



CS & IT ENGINEERING



Algorithms

Heap Algorithms

Lecture No.- 01



By- Aditya sir

Recap of Previous Lecture



Topic

Topic

Graph Traversals

↳ DFS
↳ BFS
↳ Applications

Topics to be Covered



Topic

Topic

Topic

Heap Algorithms



About Aditya Jain sir

1. Appeared for GATE during BTech and secured AIR 60 in GATE in very first attempt - City topper
2. Represented college as the first Google DSC Ambassador.
3. The only student from the batch to secure an internship at Amazon. (9+ CGPA)
4. Had offer from IIT Bombay and IISc Bangalore to join the Masters program
5. Joined IIT Bombay for my 2 year Masters program, specialization in Data Science
6. Published multiple research papers in well known conferences along with the team
7. Received the prestigious excellence in Research award from IIT Bombay for my Masters thesis
8. Completed my Masters with an overall GPA of 9.36/10
9. Joined Dream11 as a Data Scientist
10. Have mentored 12,000+ students & working professions in field of Data Science and Analytics
11. Have been mentoring & teaching GATE aspirants to secure a great rank in limited time
12. Have got around 27.5K followers on Linkedin where I share my insights and guide students and professionals.



Telegram Link for Aditya Jain sir: https://t.me/AdityaSir_PW

TTTe



Topic : Heap Algorithms

45%



#Q. In a DF-traversal of a graph 'a' with n -vertices, ' k ' edges are marked as tree edges, the number of connected components of ' G ' is:

DF

$$|V| = n$$

$$|E| = k$$

A

k

Comp Topper:-
 $\rightarrow (n-1) \times$

B

$k+1$

$\rightarrow (n-1)+1 = n \times$

C

$(n-k-1)$

$\rightarrow (n - (n-1) - 1) = 0 \times$

D

$n-k$

$\rightarrow n - (n-1) = 1 \checkmark$

Connected graph

DF



#Components = 1

$$k = n-1$$



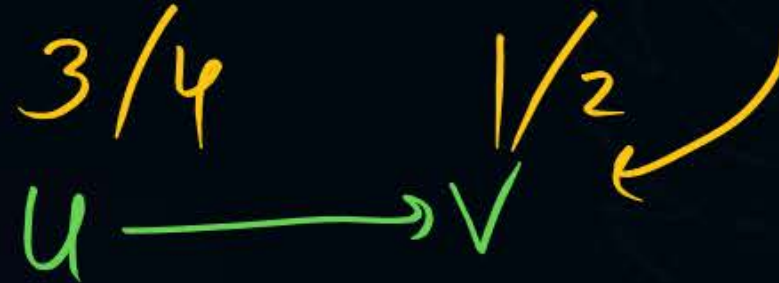
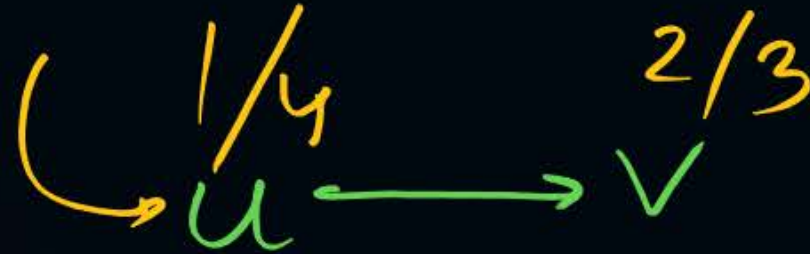
Topic : Heap Algorithms



#Q. DFS is performed on a directed acyclic graph. $D(u)$ is discovery time and $f(u)$ is finishing time. Which is true for all edges (u, v) in the graph?

mcq

28%



ans: D

A $d[u] < d[v]$ ✗

B $d[u] < f[v]$ ✗

C $f[u] < f[v]$ ✗

D $f[u] > f[v]$ ✓



Topic : Heap Algorithms



PYQ

52%

#Q. Consider an undirected graph (unweighted). If BFS of G is done from a node 'r' let $d(r, u)$ and $d(r, v)$ be the lengths of the shortest paths from r to u and v. If 'u' is visited before 'v', during the traversal, then which is true? *respectively*

- A** $d(r, u) < d(r, v)$ ✗
- B** $d(r, u) > d(r, v)$ ✗
- C** $d(r, u) \leq d(r, v)$ ✓
- D** None ✗

C



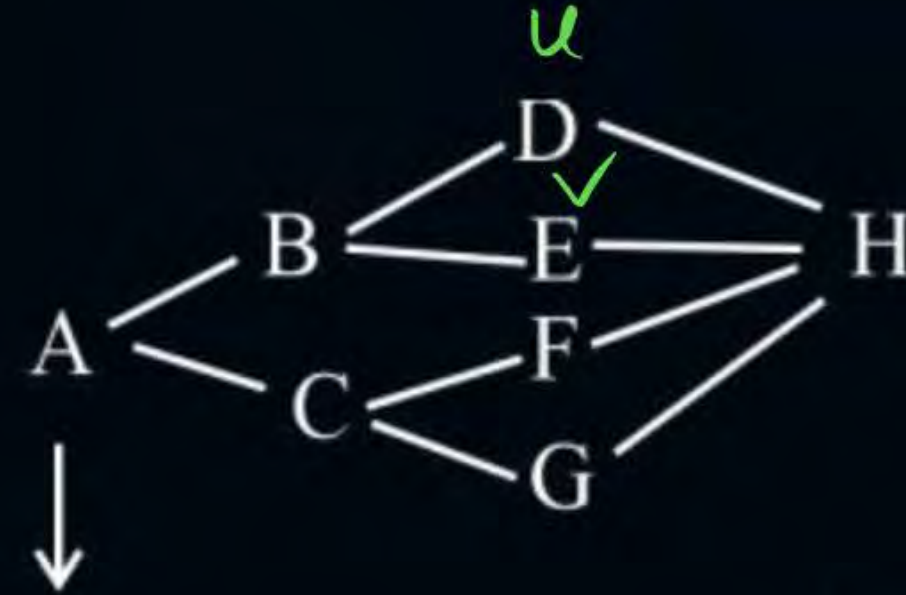
Topic : Heap Algorithms

Example:

$A \rightarrow D : 2$

$A \rightarrow E : 2$

$$d(r, u) = d(r, v)$$



Start

FIFO Queue: ✓

Node	B	C	D	E	F	G	H
Parent	A	A	B	B	C	C	D



Topic : Heap Algorithms



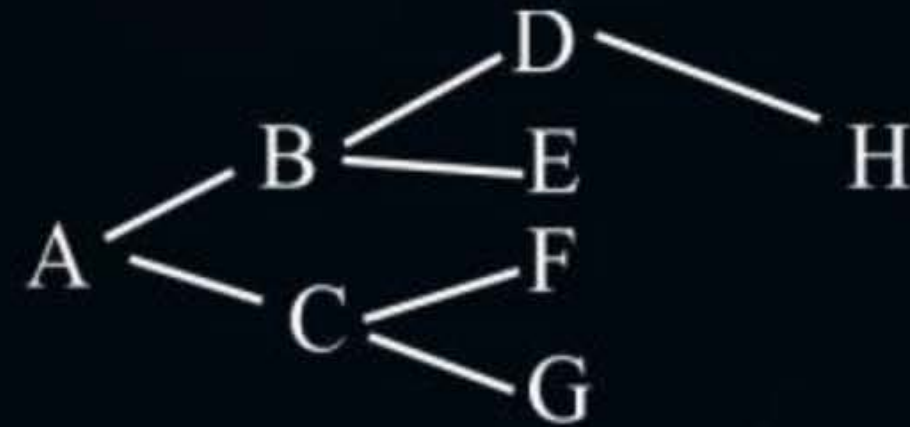
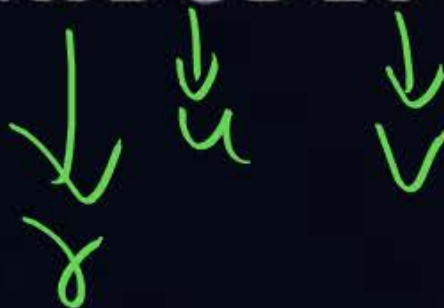
BFS at A:

$A \rightarrow C : 1$ ✓

$A \rightarrow F : 2$ ✓

$d(r, u) < d(r, v)$

o/p : A B C D E F G H





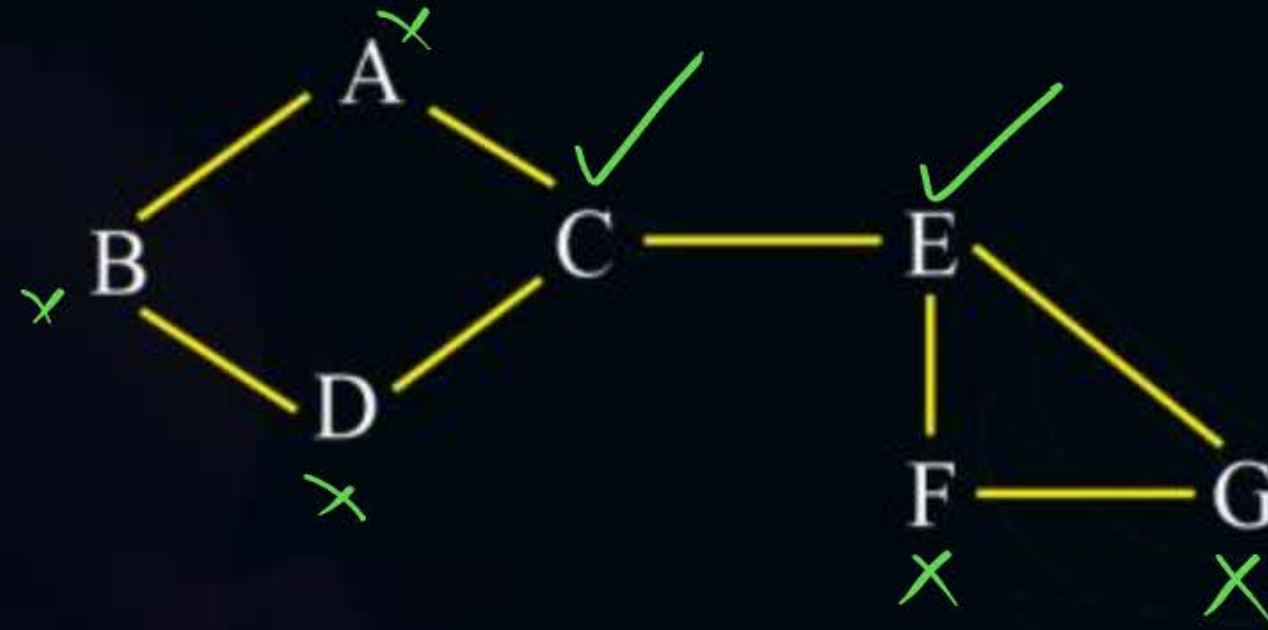
Topic : Heap Algorithms

HW

#Q. Given a graph G, it has A articulation points and B bi-connected components, then what is the value of $(A)^B$?

40%

$A = 2$



$A^B = 2^3 = 8$ ✓

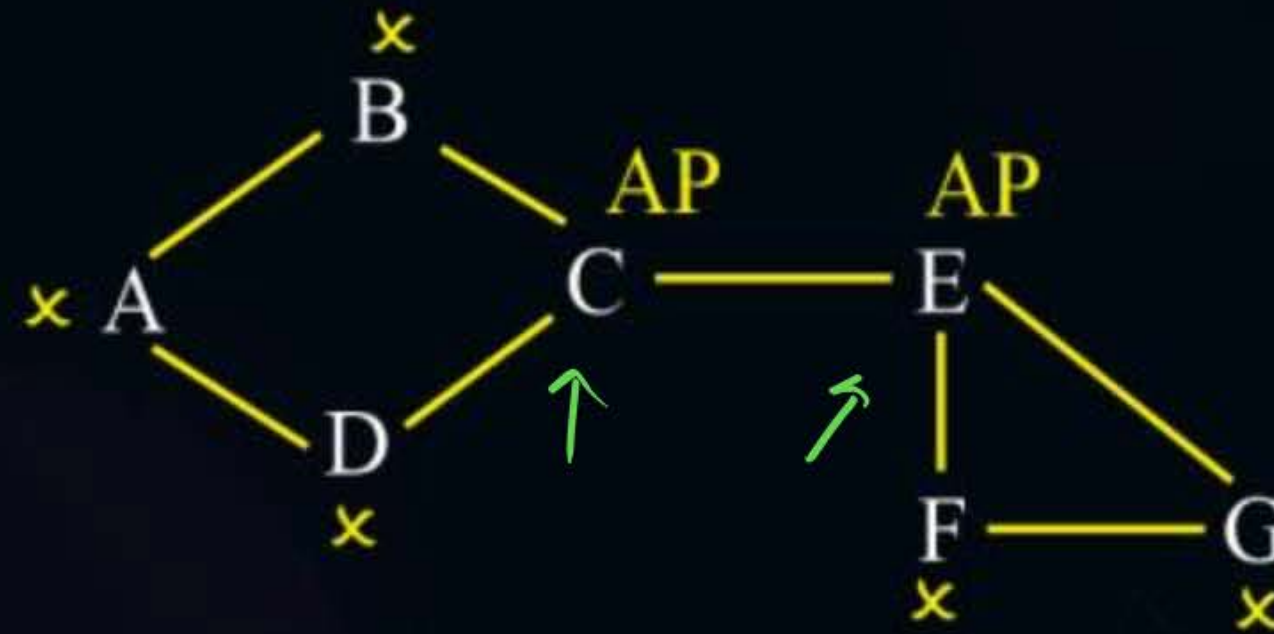


Topic : Heap Algorithms

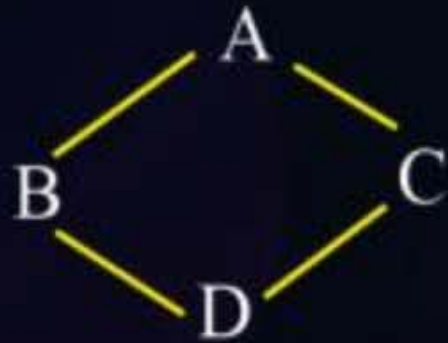


Solution:

$$\underline{\underline{B=3}}$$



BCC1



BCC2



BCC3

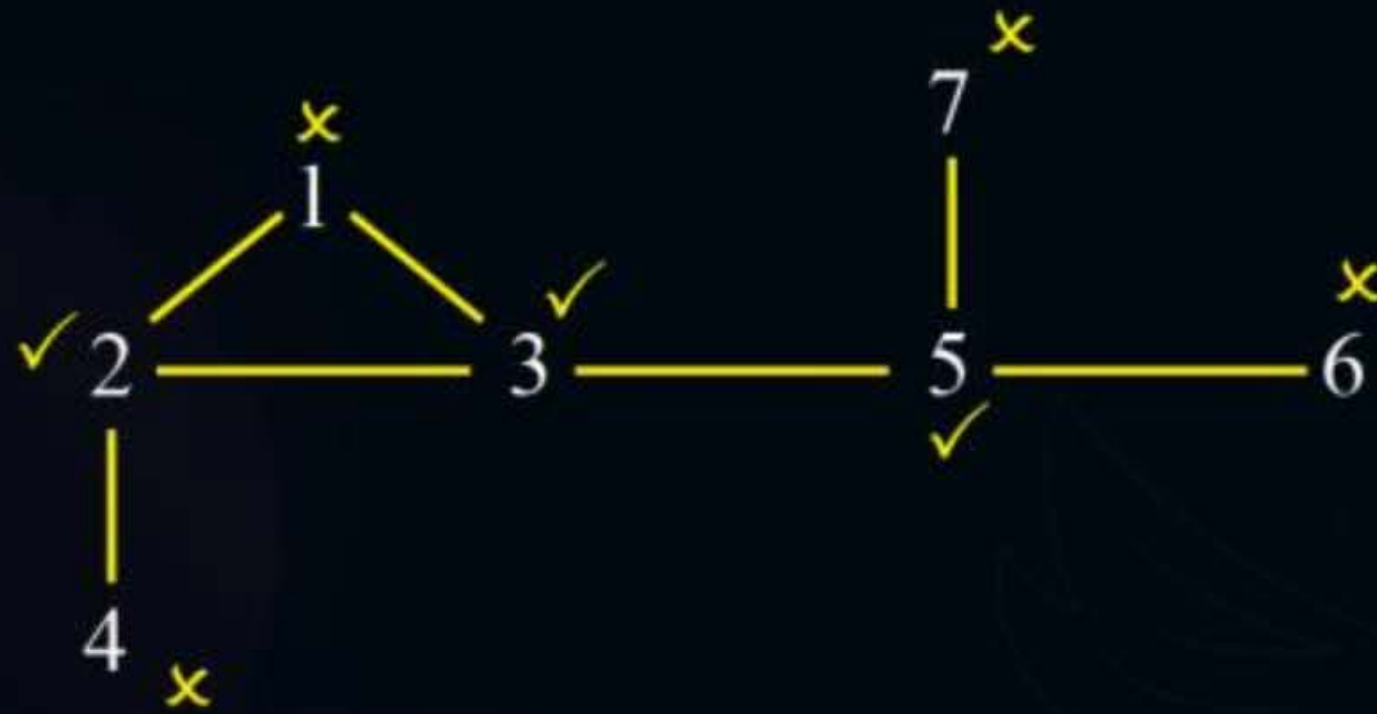




Topic : Heap Algorithms



#Q. How many Aps and what are those?



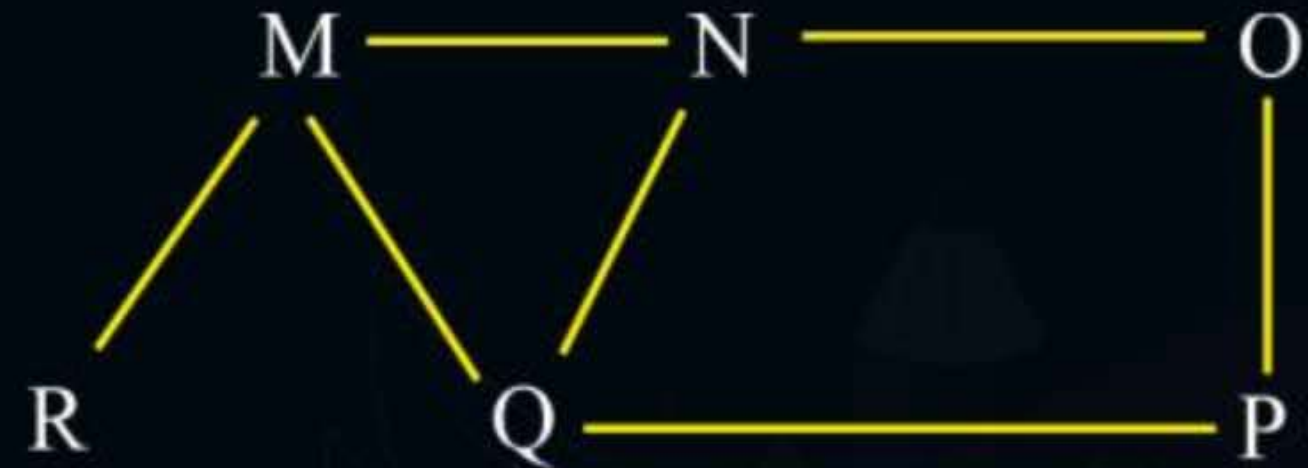
APs: 2, 3, 5
Count = 3



Topic : Heap Algorithms



#Q. Which of the following are valid BFS sequences?



A M N O P Q R *X*

B N Q M P O R *X*

C Q M N P R O ✓

D Q M N P O R *→ 70% X*
E) None.

C



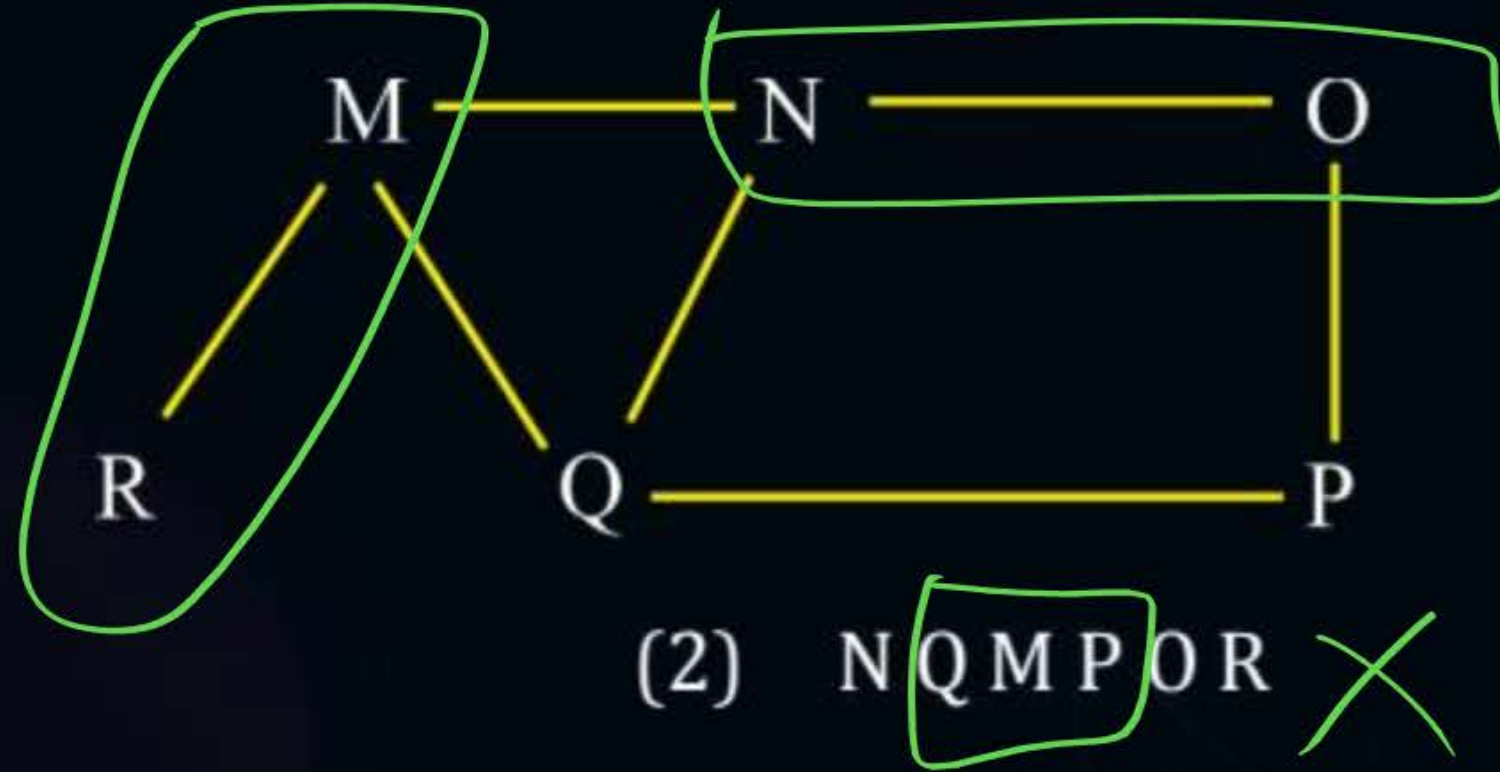
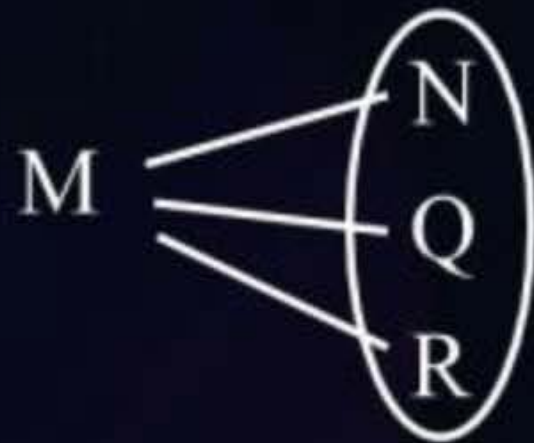
Topic : Heap Algorithms



Solution:



(1) ~~M~~ NOPQR

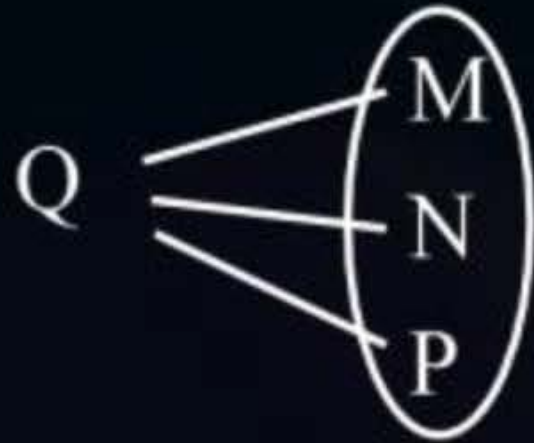




Topic : Heap Algorithms



(3) Q M N P R O



(4) Q M N P O R





Topic : Heap Algorithms

Heaps:

- Complete Binary Tree (CBT) with some additional properties.

Types of Heaps:

- (1) Max Heap (default)
- (2) Min Heap

Binary
Heap



Topic : Heap Algorithms



(1) Max Heap (default):

- A max-Heap is a Complete Binary Tree with a property that the value at each node is greater than or equal to all of its children.



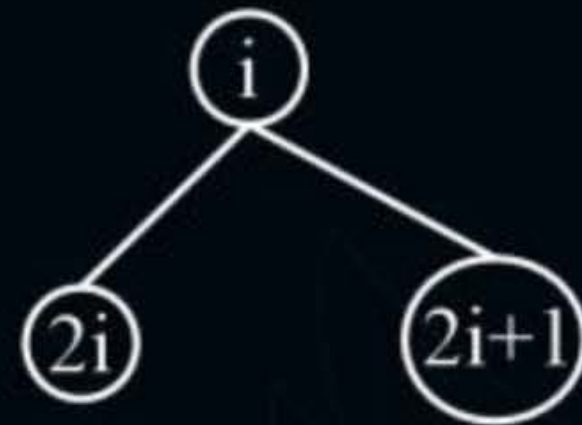
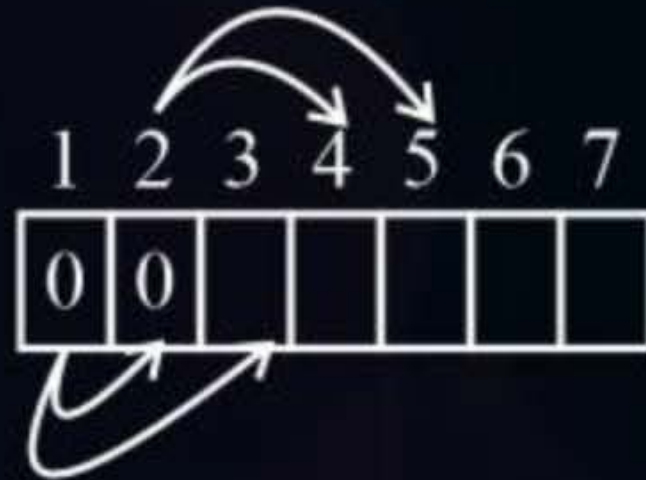


Topic : Heap Algorithms



Array representation of a Complete Binary Tree:

$i \geq 1$ (1 based indexing)





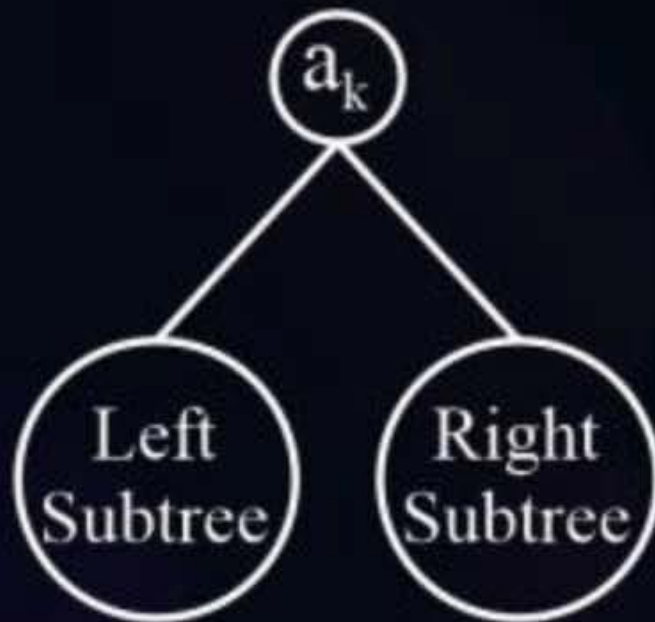
Topic : Heap Algorithms



(1) Max Heap (default):

$a_k \geq$ all values in the left subtree and the right subtrees.

General:



Complete BT (CBT)



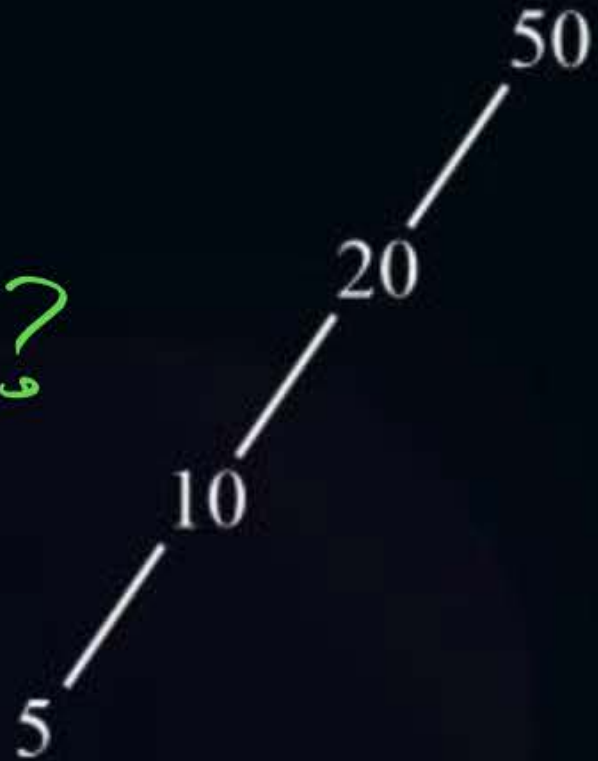


Topic : Heap Algorithms



Example 1:

max Heap ?



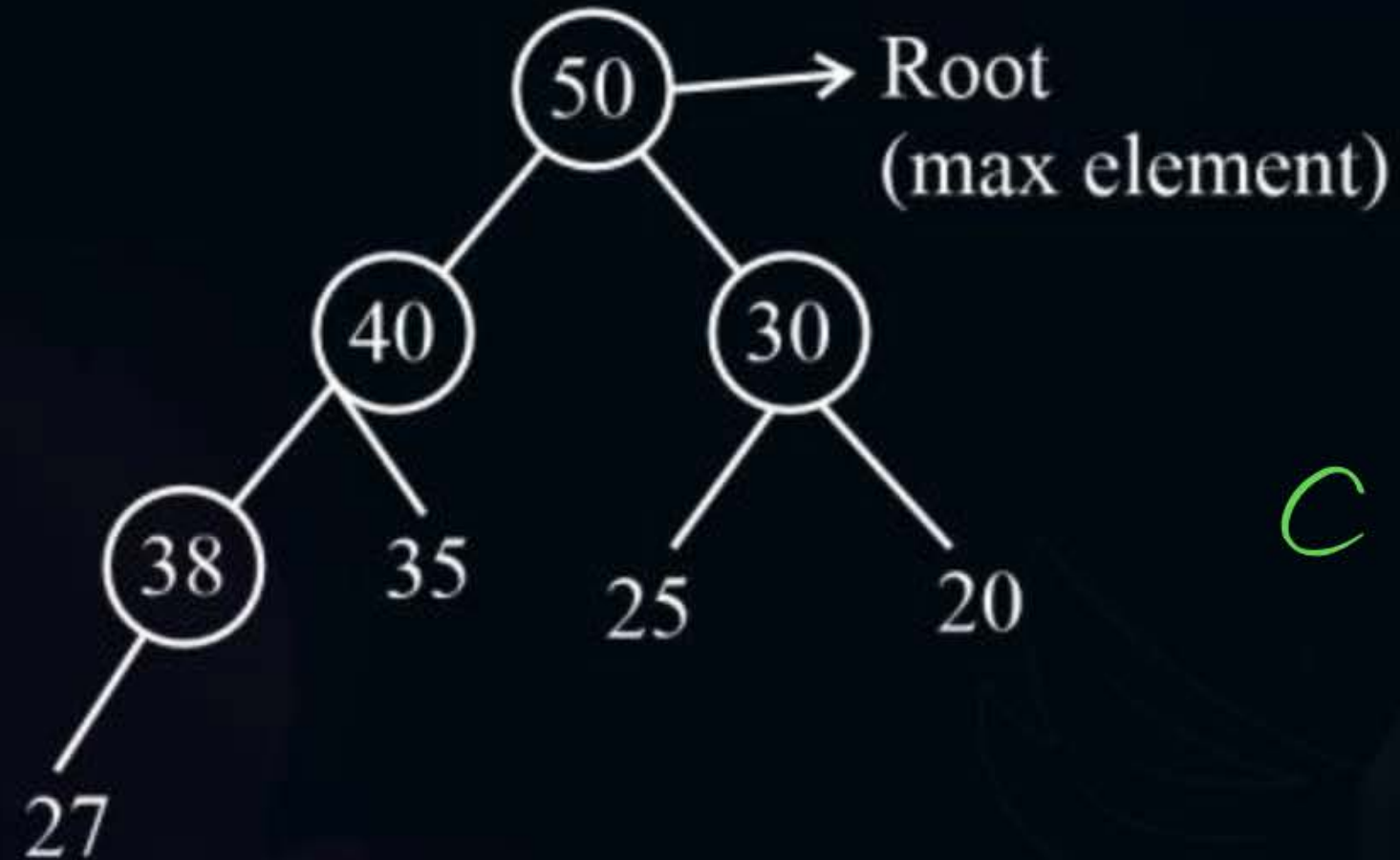
Not a Complete Binary Tree



Topic : Heap Algorithms



Example 2:



CBT ✓

Valid Max Heap ✓



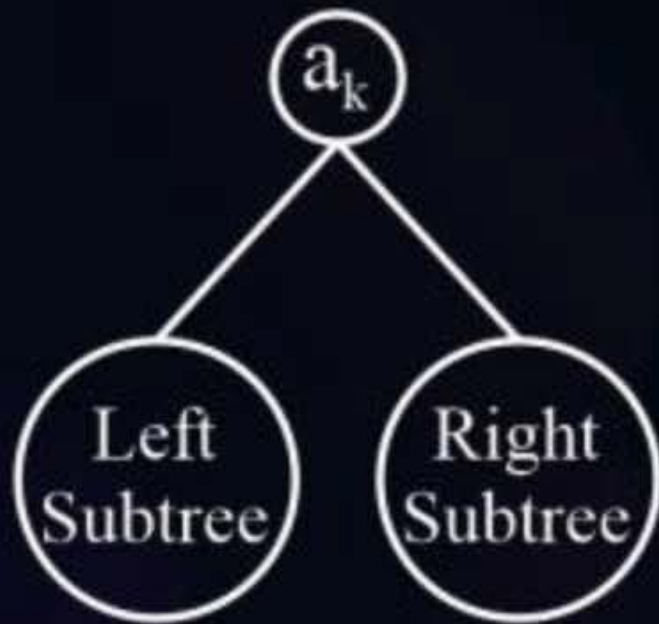
Topic : Heap Algorithms



2) Min Heap:

$a_k \leq$ all values in the left subtree and the right subtrees.

General:

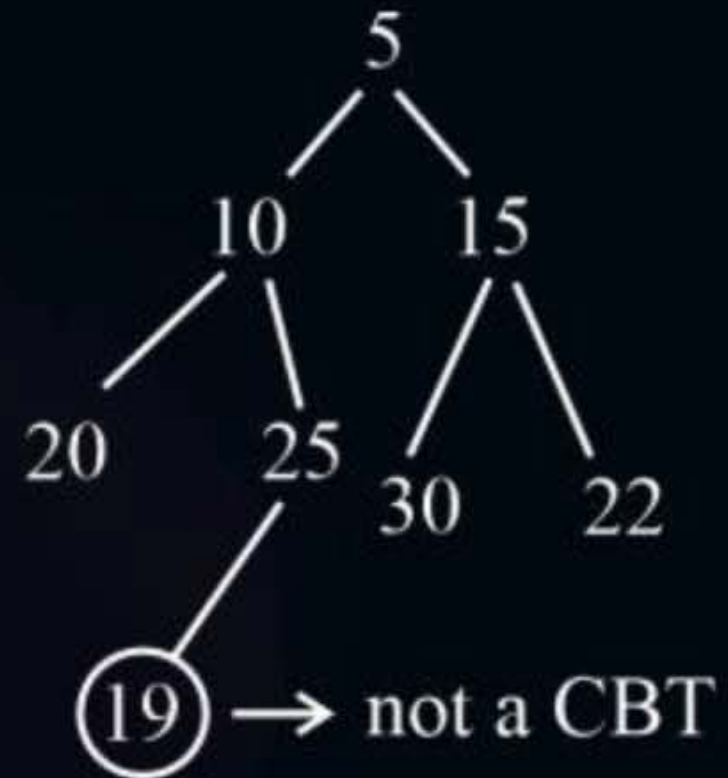




Topic : Heap Algorithms



Example 1:

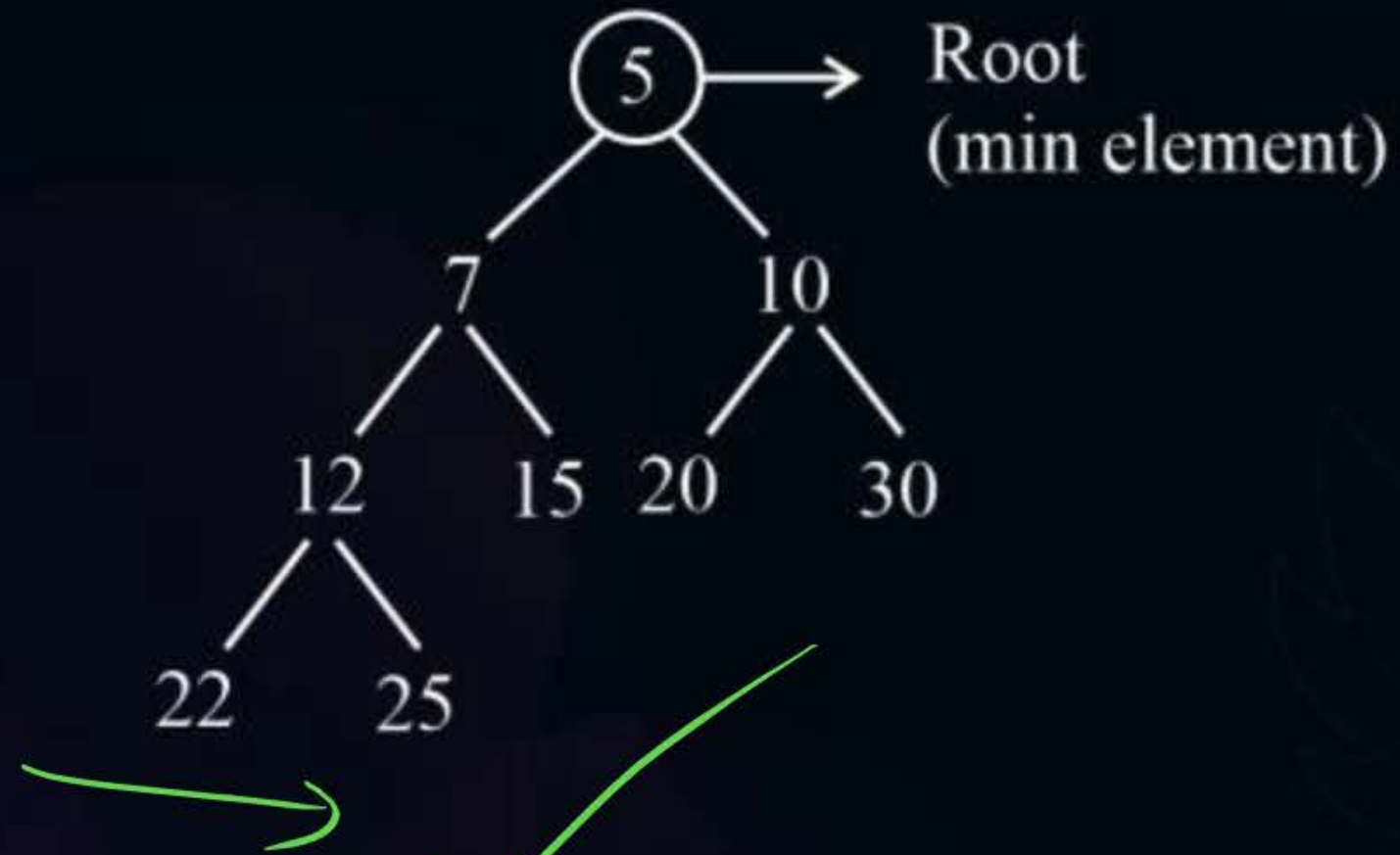


Not a Complete Binary Tree



Topic : Heap Algorithms

Example 2:





Topic : Heap Algorithms



Construction of a Heap:

- (1) Insertion method
- (2) Heapify/Build Heap method





Topic : Heap Algorithms



Insertion method:

Steps:

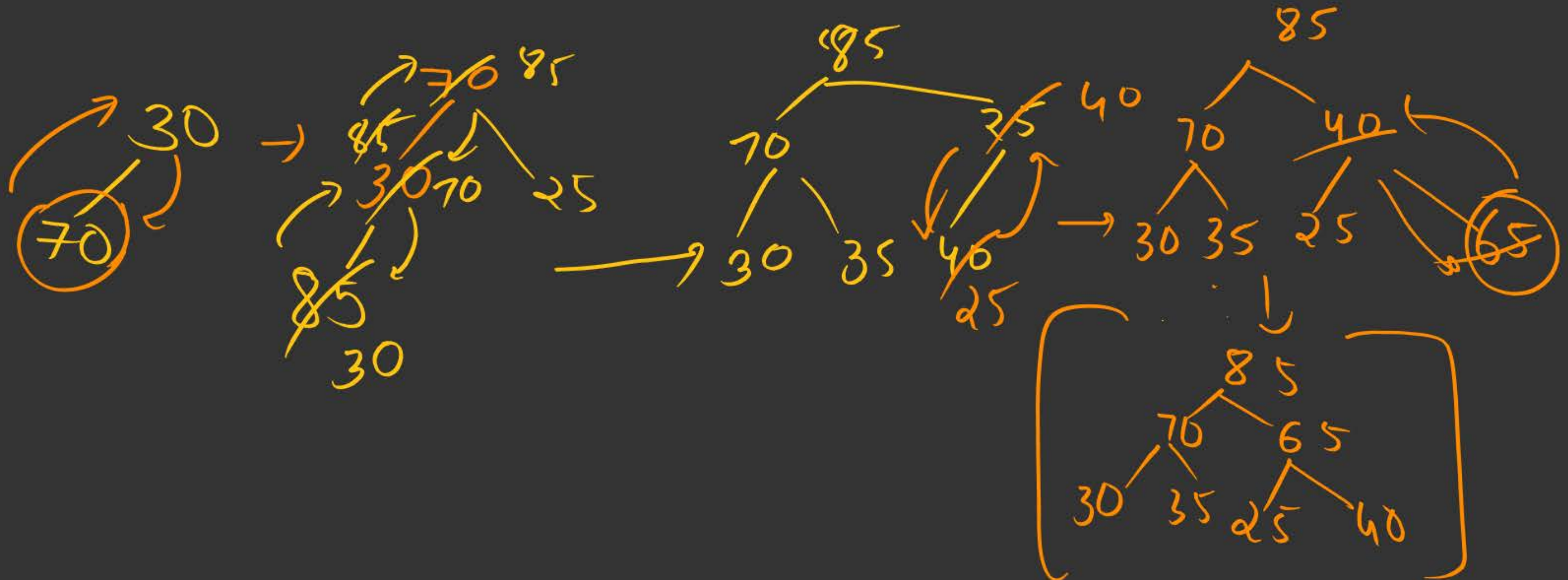
- (1) Start with an empty tree.
- (2) Insert one element at a time, ^{in CBT way} always to a valid Heap of elements that are inserted ^{ed} so far.

(A new element is ~~inserted~~ always to a valid Heap)

^{inserted}

→

eg - A: [30, 70, 25, 85, 35, 40, 65]

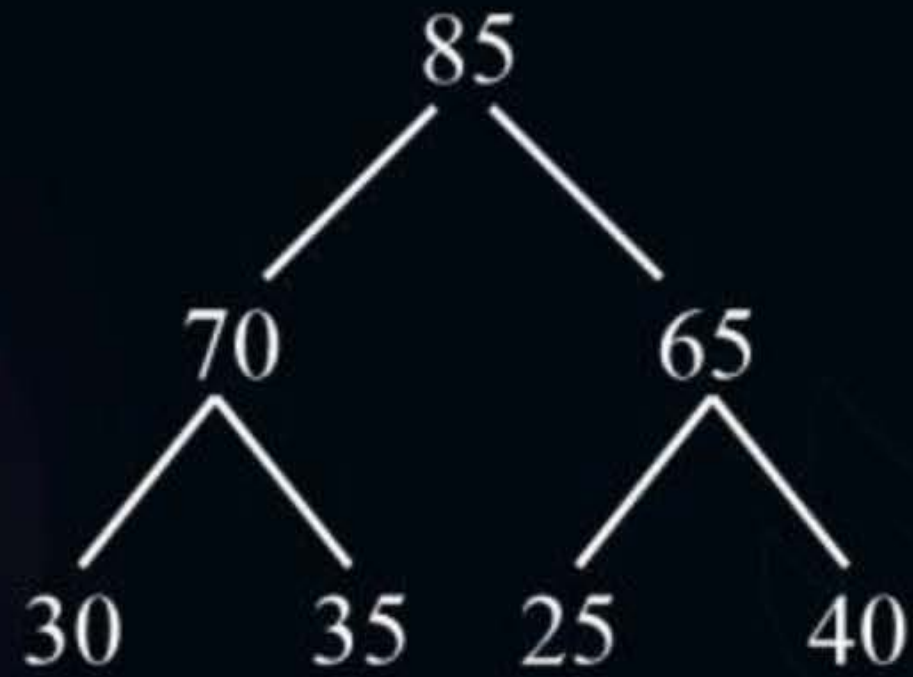




Topic : Heap Algorithms



Final Max Heap:





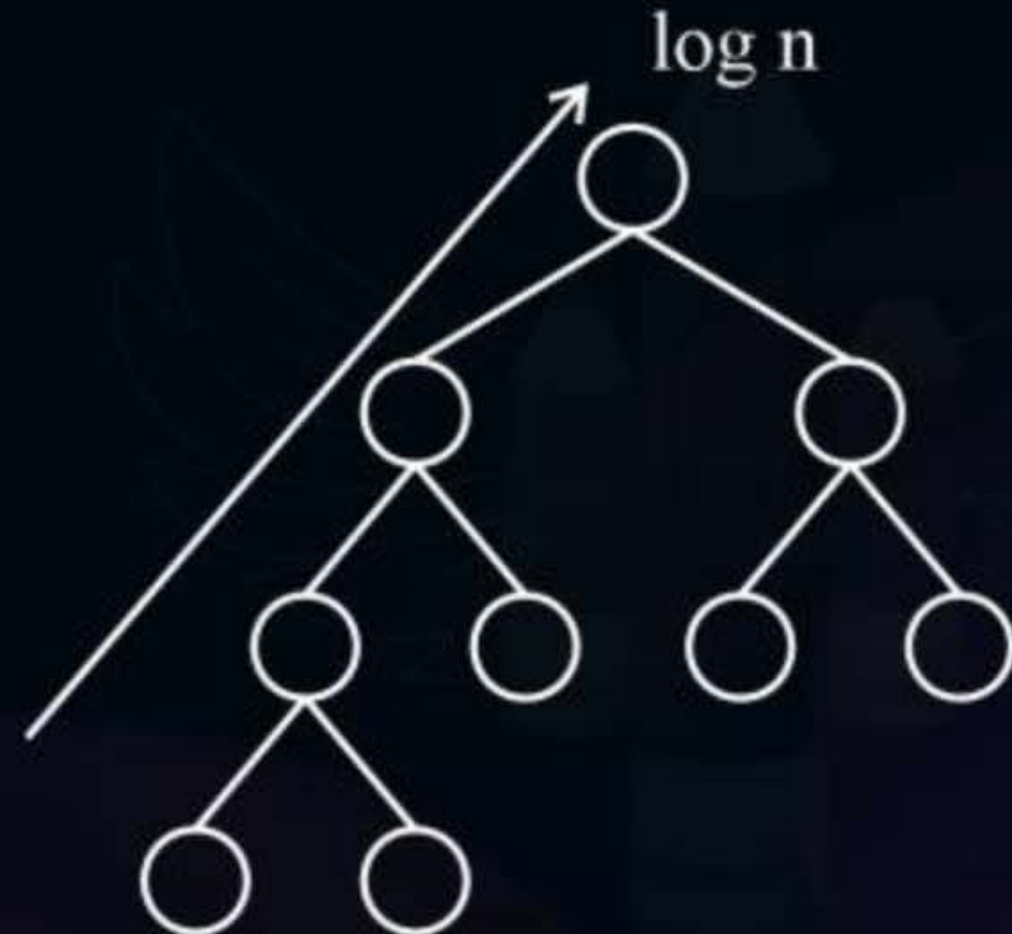
Topic : Heap Algorithms



Important Note:

Time Complexity of insert Operation:

- Given a Heap of 'n' elements, the time complexity of inserting an element into this Heap is $O(\log n)$.
- Height of a CBT of n elements
→ always $O(\log n)$





Topic : Heap Algorithms



Time Complexity of Heap creation using Insertion method:

(1) Best Case:

- I/p array already sorted in decreasing order.
- Every element will need only 1 comparison (with its parent)

$$\rightarrow n * 1 = \underline{O(n)} = \Omega(n)$$



Topic : Heap Algorithms



Time Complexity of Heap creation using Insertion method:

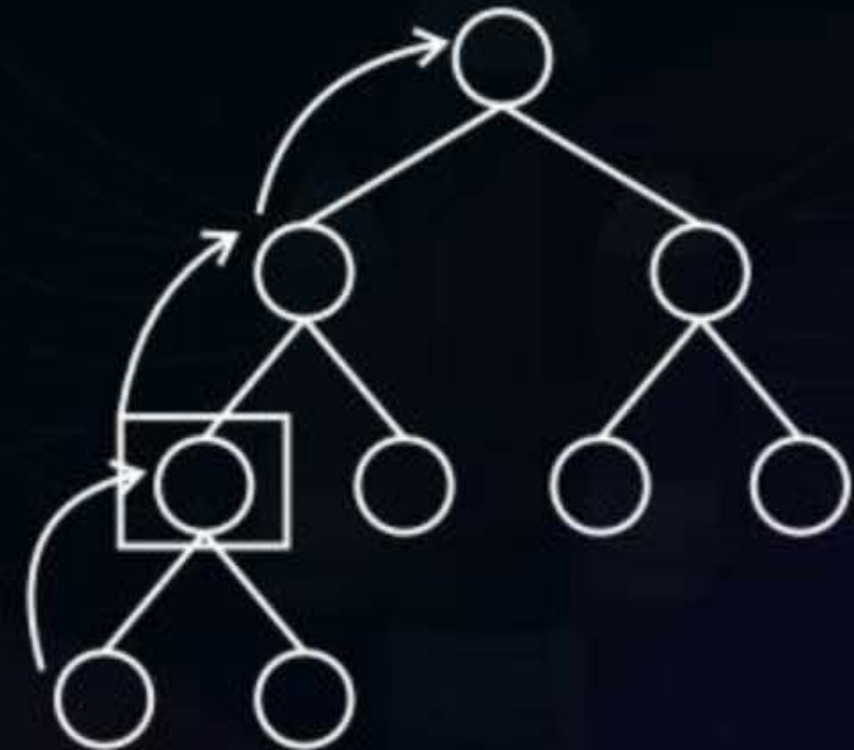
(2) Worst Case:

- I/p is already sorted in increasing order.
- Every element will move to the top, level-by-level.

$$\rightarrow O(\log_2 n)$$

Hence, for n elements,

$$\text{Overall TC} = O(n * \log_2 n)$$





Topic : Heap Algorithms

Bonus:

Logical/mathematical approach to determine worst case complexity of Heap Creation using Insertion method.



Topic : Heap Algorithms



(1) Max no. of nodes at any level 'i' of a Binary Tree (Root at level 1) = $2^{(i-1)}$

$$1 \rightarrow 1 \rightarrow 2^0$$

$$2 \rightarrow 2 \rightarrow 2^1$$

$$3 \rightarrow 4 \rightarrow 2^2$$

$$4 \rightarrow 8 \rightarrow 2^3$$





Topic : Heap Algorithms

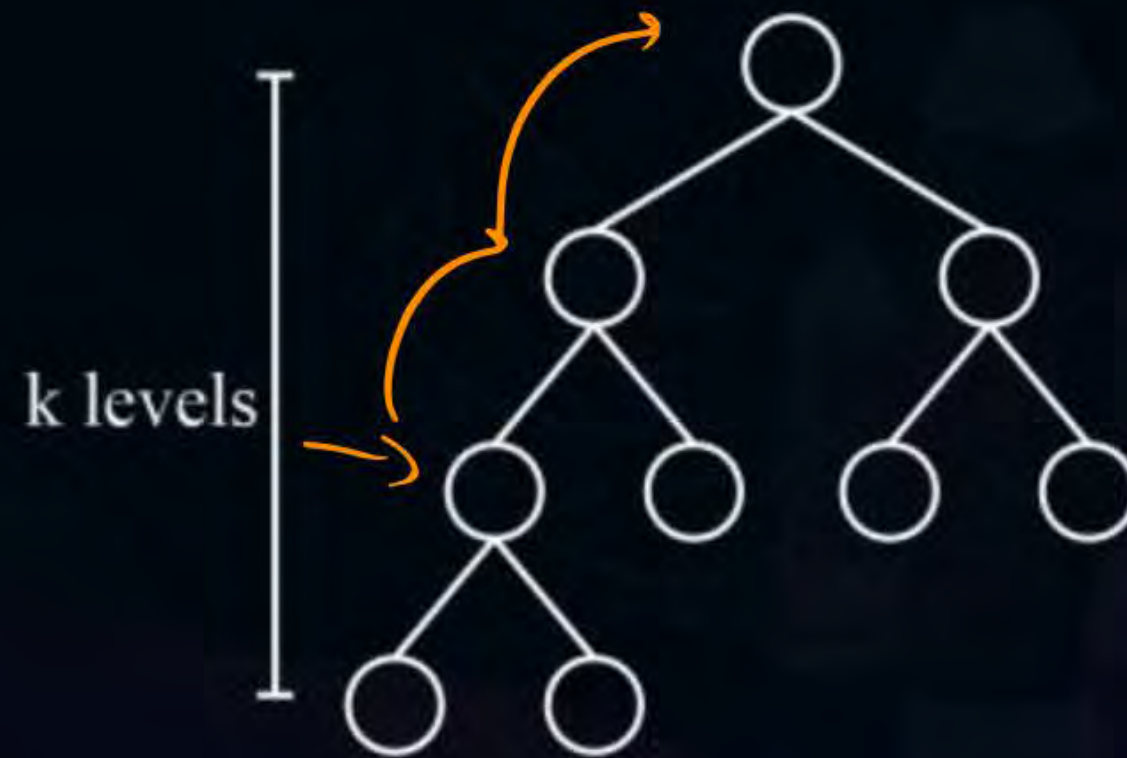


(2) The no. of level comparisons for a node that is getting inserted at level i

$$= (i - 1)$$

(3) The total no. of level comparisons for all nodes at level 'i'.

$$= 2^{(i-1)} * \underline{\underline{(i - 1)}}$$





Topic : Heap Algorithms



(4) Overall TC to create a Heap of 'n' elements

= No. of comparisons for all the nodes at all the level (assume k levels)

$$\sum_{i=1}^k 2^{(i-1)} * (i-1) = \underbrace{\sum_{i=1}^k i * 2^{(i-1)}} - \underbrace{\sum_{i=1}^k 2^{(i-1)}}$$

$k \rightarrow$ no. of levels

$$= \frac{1}{2} \left[\sum_{i=1}^k i * 2^i - \sum_{i=1}^k 2^i \right]$$



Topic : Heap Algorithms



Important:

$$\sum_{i=1}^k i * 2^i = (k - 1) * 2^{(k+1)} + 2 \quad \dots (i)$$

$$\sum_{i=1}^k 2^i = \frac{a(r^n - 1)}{r - 1} = \frac{2(2^k - 1)}{2 - 1} = 2(2^k - 1) \quad \dots (ii)$$



Topic : Heap Algorithms



Total Comparisons

$$= \frac{1}{2} \left[\sum_{i=1}^k i * 2^i - \sum_{i=1}^k 2^i \right]$$

using Results from (i) and (ii)

$$= \frac{1}{2} \left[\left((k-1) * 2^{(k+1)} + 2 \right) - \left(2(2^k - 1) \right) \right]$$

$$= \frac{1}{2} \left[2 \times 2^k \times k - 2 \times 2^k + 2 - 2 \times 2^k + 2 \right]$$



Topic : Heap Algorithms



$$= \frac{1}{2} [2 \times 2^k \times k - 2 \times 2 \times 2^k + 2 \times 2]$$

$$= [2^k \times k - 2 \times 2^k + 2] \rightarrow O(k * 2^k)$$

$$n = 2^k \Rightarrow \underline{k = \log_2 n} \Rightarrow O(n * \log_2 n)$$





Topic : Heap Algorithms



Procedure INSERT(A, n)

Integer i, j, n;

$j = n$; $i \leftarrow \lfloor n/2 \rfloor$; item $\leftarrow A(n)$

while $i > 0$ and $A(i) < \text{item}$ do

$A(j) \leftarrow A(i)$ // move the parent down //

$j \leftarrow i$; $i \leftarrow \lfloor i/2 \rfloor$

repeat

$A(j) \leftarrow \text{item}$ // a place for $A(n)$ is found //

end INSERT

for $i \leftarrow 2$ to n do

call INSERT (A, i)

repeat

Idea



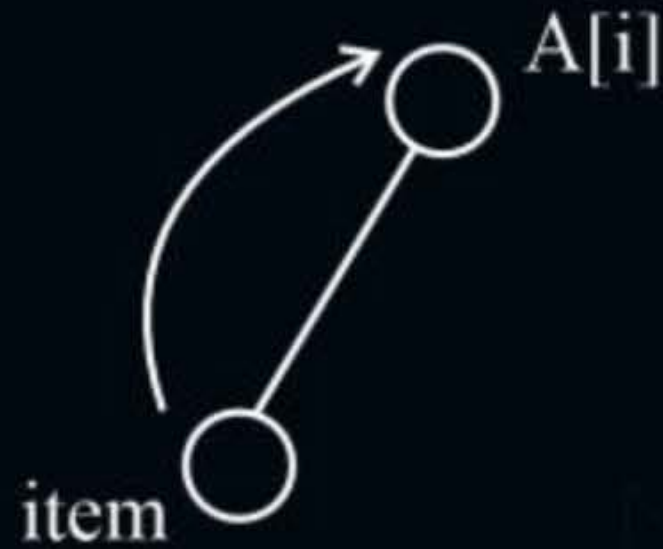


Topic : Heap Algorithms



Heap Creation using Insertion method:

- $A[i] < \text{item}$
- $\text{parent} < \text{child}$
swap





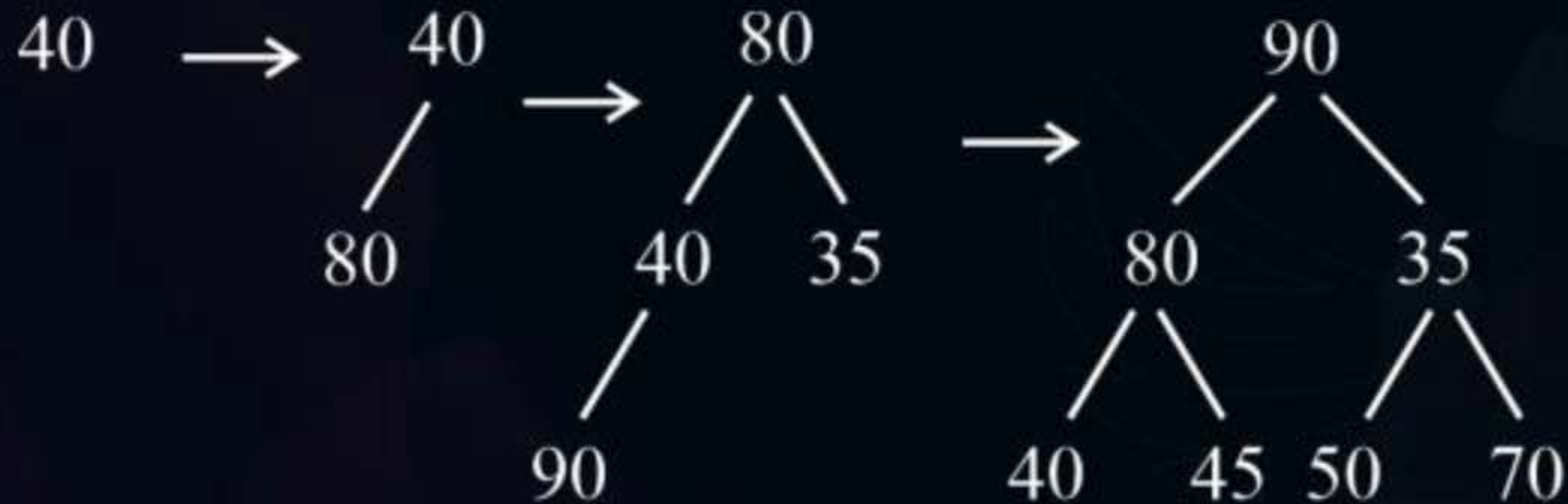
Topic : Heap Algorithms

Test: Given A : [40, 80, 35, 90, 45, 50, 70]

HW

#Q. Create a max-Heap using insertion method.

How many swaps happened during the process?



Total swaps = 1 + 2 + 1 + 1 = 5



2 mins Summary



Topic

Topic

Topic

Topic

Topic

PYQs on Graph

Intro to Heaps



THANK - YOU